MANUAL DA ENTREGA FINAL

Projeto Integrador — Modelagem de Dados

Do Modelo Conceitual ao Banco de Dados Implementado

1. QUAL É O DESAFIO DESTA ENTREGA?

Na primeira entrega, sua equipe partiu de um problema real de uma micro ou pequena empresa e percorreu o seguinte caminho:

EMPRESA → PROCESSOS → PROBLEMAS → REQUISITOS → REGRAS DE NEGÓCIO → ENTIDADES → ATRIBUTOS → RELACIONAMENTOS → CARDINALIDADES → DER

Agora o desafio muda.

Vocês já identificaram **o que o negócio precisa** e construíram uma primeira representação conceitual dos dados.

Na entrega final, a equipe deverá transformar esse modelo em uma **estrutura de banco de dados organizada, normalizada, implementada e capaz de produzir informações úteis para a empresa**.

O novo caminho será:

DER

↓

REVISÃO DO MODELO CONCEITUAL

↓

MODELO LÓGICO

↓

PK + FK + RESTRIÇÕES

↓

NORMALIZAÇÃO — 1FN → 2FN → 3FN

↓

DICIONÁRIO DE DADOS ATUALIZADO

↓

MODELO FÍSICO

↓

SQL

↓

MASSA DE DADOS

↓

CONSULTAS

↓

INFORMAÇÃO GERENCIAL

↓

POSSIBILIDADES DE BI E IA

A pergunta central desta etapa será:

“O banco de dados que projetamos representa corretamente o negócio e funciona de maneira consistente?”

2. ESTA ENTREGA SUBSTITUI A PRIMEIRA?

NÃO.

A entrega final é uma **evolução da primeira entrega**.

Isso significa que o grupo deverá revisar o trabalho anteriormente desenvolvido e realizar as correções necessárias.

Se, durante a construção do modelo lógico, vocês descobrirem que:

- uma entidade está incorreta;
- falta uma entidade;
- existe um relacionamento inadequado;
- uma cardinalidade precisa ser corrigida;

- uma regra de negócio não está clara;
- um atributo está associado à entidade errada;

o grupo deverá **corrigir o modelo**.

Não mantenham um erro apenas porque ele estava presente na primeira entrega.

Um projeto de banco de dados evolui.

Corrigir uma decisão após analisá-la novamente faz parte do processo de modelagem.

3. O QUE VOCÊS PRECISAM ENTREGAR?

A entrega final deverá apresentar, de maneira organizada:

1. Revisão da primeira entrega

- correções necessárias;
- atualização do DER;
- atualização das regras que sofreram alterações;
- justificativas das principais mudanças.

2. Modelo Lógico Relacional

Deverá conter:

- tabelas;
- atributos;
- chaves primárias — PK;
- chaves estrangeiras — FK;
- chaves compostas, quando necessárias;
- relacionamentos convertidos;
- restrições;
- integridade referencial.

3. Normalização

A equipe deverá demonstrar a aplicação de:

- 1FN;
- 2FN;
- 3FN;
- dependências funcionais relevantes;
- justificativas das decomposições realizadas.

4. Dicionário de Dados atualizado

O dicionário deverá acompanhar a evolução do modelo e representar a estrutura efetivamente implementada.

5. Script SQL completo

Deverá contemplar:

- `CREATE DATABASE`;
- `CREATE TABLE`;
- PK;
- FK;
- constraints;
- `INSERT`;
- `SELECT`;
- `JOIN`;
- `UPDATE`;
- `DELETE`;
- consultas gerenciais;
- consultas de auditoria.

6. Massa de Dados

Dados suficientes e coerentes para testar:

- relacionamentos;
- restrições;
- consultas;
- processos empresariais;
- integridade do banco.

7. Repositório GitHub

Deverá reunir:

- documentação;
- diagramas;
- scripts;
- dicionário de dados;
- histórico de commits;
- evidências da evolução e colaboração.

8. Análise de dados e possibilidades de evolução

A equipe deverá demonstrar como os dados armazenados poderão apoiar:

- indicadores;
- informações gerenciais;
- tomada de decisão;
- BI;

- futuras aplicações de IA.

9. Apresentação e defesa final

A apresentação seguirá manual específico disponibilizado pelo professor.

4. COMO REALIZAR A ENTREGA FINAL?

Siga as etapas abaixo na ordem.

ETAPA 1 — REVISEM A PRIMEIRA ENTREGA

Antes de criar tabelas ou escrever SQL, retornem ao trabalho anterior.

Perguntem:

“Nosso DER ainda representa corretamente o negócio?”

Revisem:

☐ requisitos;

☐ regras de negócio;

☐ entidades;

☐ atributos;

☐ relacionamentos;

☐ cardinalidades;

☐ entidades associativas;

☐ possíveis entidades fracas.

Não comecem pelo SQL.

O SQL deverá ser consequência do modelo.

A lógica continua sendo:

NEGÓCIO → MODELO → BANCO

e não:

SQL → tentar descobrir depois qual modelo foi construído.

ETAPA 2 — REGISTREM AS PRINCIPAIS ALTERAÇÕES

Se o modelo foi alterado depois da primeira entrega, registrem as mudanças relevantes.

Uma forma simples é:

O que foi alterado?	Como era?	Como ficou?	Por que alteramos?
Cardinalidade Cliente–Pedido	1:1	1	Um cliente pode realizar vários pedidos
Produto–Fornecedor	1	N	Um produto pode possuir vários fornecedores
Endereço	atributo	tabela	Foi necessário armazenar vários endereços

Não é necessário documentar pequenas alterações estéticas.

Registrem aquilo que representa **mudança de decisão de modelagem**.

ETAPA 3 — CONVERTAM O DER PARA O MODELO LÓGICO

Agora vocês deixarão o nível conceitual e começarão a definir como os dados serão organizados no banco relacional.

Regra básica

ENTIDADE → TABELA

Exemplo:

CLIENTE

poderá originar:

`CLIENTE`

- id_cliente
- nome
- cpf

- email

Mas converter o modelo não significa apenas trocar a palavra “entidade” por “tabela”.

Será necessário definir:

TABELA → ATRIBUTOS → PK → FK → RESTRIÇÕES

ETAPA 4 — DEFINAM AS CHAVES PRIMÁRIAS

Para cada tabela, perguntem:

“Como identificaremos cada registro de maneira única?”

Exemplo:

CLIENTE

`id_cliente` — PK

PRODUTO

`id_produto` — PK

A chave primária deverá:

- identificar unicamente o registro;
- não admitir duplicidade;
- ser coerente com a estrutura da tabela.

Atenção

Não escolham uma PK apenas porque “todo mundo usa”.

Saibam responder:

“Por que esta é uma boa chave para esta tabela?”

ETAPA 5 — CONVERTAM OS RELACIONAMENTOS

Agora revisem os relacionamentos existentes no DER.

Perguntem:

“Como este relacionamento aparecerá nas tabelas?”

Em muitos casos, isso resultará na criação de uma **chave estrangeira — FK**.

Exemplo:

CLIENTE 1 PEDIDO

poderá resultar em:

CLIENTE

`id_cliente` PK

PEDIDO

`id_pedido` PK
`id_cliente` FK

A FK deverá representar corretamente a regra de negócio.

ETAPA 6 — TRATEM OS RELACIONAMENTOS N

Um relacionamento N do modelo conceitual normalmente precisará ser resolvido no modelo relacional por meio de uma **tabela associativa**.

Exemplo:

PEDIDO N PRODUTO

Não será implementado diretamente dessa maneira.

Poderemos ter:

PEDIDO

`id_pedido` PK

PRODUTO

`id_produto` PK

ITEM_PEDIDO

`id_pedido` PK/FK
`id_produto` PK/FK

```
quantidade
valor_unitario
```

Neste exemplo:

ITEM_PEDIDO possui chave primária composta.

Perguntem:

O relacionamento possui informações próprias?

Quantidade, valor praticado, desconto ou outras informações podem pertencer à associação e não exclusivamente a Pedido ou Produto.

ETAPA 7 — VERIFIQUEM ENTIDADES FRACAS

Revisem o modelo e identifiquem situações nas quais uma entidade depende de outra para existir ou para ser identificada no contexto do negócio.

Não confundam:

ENTIDADE ASSOCIATIVA

normalmente surge para representar uma associação, especialmente em relacionamentos N.

com:

ENTIDADE FRACA

possui dependência de existência ou identificação em relação a outra entidade.

A existência de uma entidade fraca deverá ser consequência da regra de negócio, e não uma exigência artificial para “ter uma entidade fraca no projeto”.

ETAPA 8 — APLIQUEM A 1FN

Pergunta:

“Cada campo armazena adequadamente um único valor?”

Evitem estruturas como:

id_cliente	nome	telefones
1	Ana	9999-1111, 9888-2222

Perguntem:

- existem listas dentro de campos?
- existem grupos repetitivos?
- existem colunas como telefone1, telefone2, telefone3?
- um atributo está tentando armazenar várias ocorrências?

Regra prática da 1FN:

“UM CAMPO → UM VALOR”

ETAPA 9 — APLIQUEM A 2FN

Atenção especial às relações que possuem **chaves candidatas compostas**.

Perguntem:

“Cada atributo não chave depende da CHAVE TODA?”

Exemplo:

MATRICULA

PK = `(id_aluno, id_disciplina)`

Se tivermos:

`nome_aluno`

e:

`id_aluno → nome_aluno`

então `nome_aluno` depende apenas de parte da chave composta.

Temos uma dependência parcial que deverá ser analisada.

Pergunta-chave da 2FN:

“DEPENDE DA CHAVE TODA?”

ETAPA 10 — APLIQUEM A 3FN

Agora procurem dependências transitivas inadequadas.

Exemplo:

DISCIPLINA

`id_disciplina` PK
`nome_disciplina`
`id_professor`
`nome_professor`

Se:

`id_disciplina → id_professor`

e:

`id_professor → nome_professor`

então `nome_professor` não deveria estar armazenado dessa forma em DISCIPLINA.

Poderemos ter:

PROFESSOR

`id_professor` PK
`nome_professor`

DISCIPLINA

`id_disciplina` PK
`nome_disciplina`
`id_professor` FK

Pergunta prática da 3FN:

“O atributo depende diretamente da chave ou depende de outro atributo?”

ETAPA 11 — DOCUMENTEM A NORMALIZAÇÃO

Não escrevam simplesmente:

“O banco foi normalizado até a 3FN.”

Isso não demonstra o processo realizado.

Selecionem situações relevantes do projeto e apresentem:

estrutura analisada → dependência funcional → problema identificado → correção realizada

Exemplo:

ANTES

```
DISCIPLINA(id_disciplina, nome_disciplina, id_professor, nome_professor)
```

DEPENDÊNCIAS

```
id_disciplina → id_professor

id_professor → nome_professor
```

PROBLEMA

Dependência transitiva.

DEPOIS

```
PROFESSOR(id_professor, nome_professor)

DISCIPLINA(id_disciplina, nome_disciplina, id_professor)
```

Não inventem erros apenas para mostrar uma normalização.

Documentem situações que realmente façam sentido no projeto.

ETAPA 12 — DEFINAM AS RESTRIÇÕES

Agora perguntem:

“Quais regras podem ser protegidas pelo próprio banco de dados?”

Avaliem o uso de:

- `PRIMARY KEY`;
- `FOREIGN KEY`;
- `NOT NULL`;
- `UNIQUE`;
- `CHECK`;
- `DEFAULT`.

Exemplo:

Se o CPF não pode ser repetido, a equipe deverá analisar a possibilidade de:

```
UNIQUE
```

Se uma quantidade não pode ser negativa:

```
CHECK
```

Não utilizem uma constraint apenas para dizer que utilizaram.

Toda restrição deverá ter uma justificativa.

ETAPA 13 — ATUALIZEM O DICIONÁRIO DE DADOS

O dicionário conceitual preliminar da primeira entrega deverá agora acompanhar o modelo efetivamente construído.

Para cada atributo, registrem informações suficientes para que outra pessoa consiga compreender o banco.

Sugestão:

Campo	Descrição
Tabela	Nome da tabela
Atributo	Nome do atributo
Descrição	O que o dado representa
Tipo	Tipo de dado
Tamanho	Quando aplicável
PK	Sim/Não
FK	Sim/Não
NULL	Permitido/Não permitido
UNIQUE	Sim/Não
DEFAULT	Quando existir
Regra/Restrição	Regra relevante

O dicionário deve representar o banco implementado.

Se o SQL mudou e o dicionário não mudou, existe inconsistência na documentação.

ETAPA 14 — CONSTRUAM O SCRIPT SQL

Somente depois de revisar o modelo lógico e a normalização, avancem para a implementação.

O script deverá contemplar, conforme aplicável:

```
CREATE DATABASE

CREATE TABLE

PRIMARY KEY

FOREIGN KEY

NOT NULL
```

```
UNIQUE

CHECK

DEFAULT

INSERT INTO

SELECT

JOIN

UPDATE

DELETE
```

Além de:

consultas gerenciais;

consultas de auditoria.

ETAPA 15 — TESTEM O SCRIPT

Não entreguem um script que nunca foi executado integralmente.

Façam o teste:

BANCO VAZIO

↓

EXECUTAR O SCRIPT

↓

CRIAR AS TABELAS

↓

CRIAR AS RESTRIÇÕES

↓

INSERIR OS DADOS

↓

EXECUTAR AS CONSULTAS

↓

VERIFICAR OS RESULTADOS

Perguntem:

“Outra pessoa conseguiria executar nosso projeto utilizando os arquivos entregues?”

Se a resposta for não, a entrega ainda não está pronta.

ETAPA 16 — CONSTRUAM UMA MASSA DE DADOS COERENTE

O banco deverá possuir dados suficientes para permitir testes significativos.

Não criem apenas:

- um cliente;
- um produto;
- um pedido.

A massa de dados deverá permitir testar:

- diferentes clientes;
- diferentes produtos;
- diferentes situações;
- relacionamentos;
- restrições;
- consultas;
- regras do negócio.

Os dados precisam ser coerentes entre si.

Uma grande quantidade de dados incoerentes não representa uma boa massa de testes.

ETAPA 17 — USO DE IA NA GERAÇÃO DE DADOS E SCRIPTS

Ferramentas de Inteligência Artificial poderão auxiliar na elaboração de scripts e na geração da massa de dados.

Entretanto:

A IA PODE AUXILIAR. A RESPONSABILIDADE TÉCNICA CONTINUA SENDO DO GRUPO.

Se uma IA gerar 500 registros, o grupo deverá verificar se esses registros:

- respeitam as PKs;
- respeitam as FKs;
- respeitam as constraints;
- fazem sentido para o negócio;
- não criam inconsistências.

Os prompts utilizados para auxiliar a geração em escala da massa de dados deverão ser documentados no projeto.

Nunca considerem uma resposta da IA correta apenas porque foi gerada por uma IA.

GERAR → TESTAR → VALIDAR → CORRIGIR

ETAPA 18 — CRIEM CONSULTAS GERENCIAIS

Agora façam o banco responder perguntas relevantes para a empresa.

Não pensem primeiro:

"Precisamos usar JOIN."

Pensem:

"Que informação o gestor precisa conhecer?"

Exemplos:

- quais produtos são mais vendidos?
- quais clientes mais compram?
- qual foi o volume de vendas por período?
- quais produtos possuem estoque reduzido?
- quais serviços são mais procurados?

Depois construam a consulta SQL necessária para responder à pergunta.

CONSULTA GERENCIAL = PERGUNTA DO NEGÓCIO + SQL + INFORMAÇÃO PARA DECISÃO

ETAPA 19 — CRIEM CONSULTAS DE AUDITORIA

Uma consulta de auditoria possui outra finalidade.

Ela deverá ajudar a verificar a qualidade ou consistência dos dados.

Perguntem:

“Como podemos utilizar SQL para procurar situações que merecem verificação?”

Por exemplo:

- registros sem associação esperada;
- situações potencialmente inconsistentes;
- duplicidades quando aplicável;
- dados fora de condições esperadas;
- relacionamentos que precisam ser conferidos.

Não criem consultas apenas para cumprir o item.

Expliquem:

“O que esta consulta está verificando?”

ETAPA 20 — PENSEM COMO GESTORES

Depois de construir as consultas, façam uma nova pergunta:

“Que decisões poderiam ser tomadas utilizando estes dados?”

Vocês estão deixando de olhar apenas para:

DADO

e começando a pensar em:

DADO → INFORMAÇÃO → DECISÃO

Exemplo:

Dado: vendas realizadas.

Informação: produtos mais vendidos por período.

Decisão: reorganizar estoque e compras.

ETAPA 21 — PENSEM EM BI E IA

Não será necessário implementar uma solução completa de Business Intelligence ou Inteligência Artificial.

O objetivo é demonstrar que o grupo consegue perceber o **potencial dos dados que estruturou**.

Escolham situações relevantes.

Exemplo:

BI

Pergunta: Como as vendas evoluíram nos últimos meses?

Dados: pedido + item_pedido + produto + data.

Possibilidade: dashboard de evolução das vendas.

IA

Problema: prever demanda de determinados produtos.

Dados possíveis: histórico de vendas + produto + período + quantidade.

Possibilidade futura: modelo de previsão de demanda.

Atenção

Não escrevam simplesmente:

“O banco poderá ser utilizado por uma IA.”

Expliquem:

PROBLEMA → DADOS → ANÁLISE → POSSÍVEL BENEFÍCIO

ETAPA 22 — ORGANIZEM O GITHUB

O GitHub deverá representar a evolução do projeto.

Uma sugestão de organização:

```
Projeto-ERP/
│
├── README.md
│
├── docs/
│   ├── requisitos/
│   ├── diagramas/
│   ├── dicionario-dados/
│   └── normalizacao/
│
├── sql/
│   ├── ddl/
│   ├── dados/
│   ├── consultas/
│   └── auditoria/
│
├── evidencias/
│
└── apresentacao/
```

A estrutura poderá ser adaptada.

O importante é que outra pessoa consiga localizar facilmente:

documentação + modelos + scripts + dados + consultas + evidências

ETAPA 23 — ATUALIZEM O README

O README deverá funcionar como a **porta de entrada do projeto**.

Uma possível estrutura:

1. Identificação do projeto

2. Integrantes

3. Empresa e problema

4. Solução proposta

5. Requisitos e regras de negócio

6. DER atualizado

7. Modelo lógico

8. Normalização

9. Dicionário de Dados

10. Implementação SQL

11. Massa de Dados

12. Consultas Gerenciais

13. Consultas de Auditoria

14. Potencial para BI e IA

15. Histórico/evolução do projeto

16. Conclusão

O README não deverá ser apenas uma lista de links.

Ele deverá ajudar o leitor a compreender **o que foi construído e onde encontrar cada artefato**.

24. O QUE SERÁ AVALIADO?

A avaliação final considera três dimensões:

Dimensão	Peso
Conceitual — compreender e justificar	**30%**
Procedimental — construir, implementar e demonstrar	**50%**
Atitudinal — participar, colaborar e agir profissionalmente	**20%**
TOTAL	**100%**

25. DIMENSÃO CONCEITUAL — 30%

25.1 Conversão do modelo conceitual para o lógico — 7,5%

Será observado se a equipe consegue:

- converter entidades em tabelas — **1,5%**;
- converter atributos corretamente — **1,5%**;
- converter relacionamentos — **2,0%**;
- converter cardinalidades — **2,5%**.

Pergunta orientadora:

“O modelo lógico representa corretamente aquilo que estava no DER e nas regras de negócio?”

25.2 Normalização — 7,5%

Será observado:

- 1FN — **1,5%**;
- 2FN — **2,0%**;
- 3FN — **2,5%**;
- justificativa do processo — **1,5%**.

Não basta estar normalizado.

A equipe deverá demonstrar que **compreende por que a estrutura atende às formas normais estudadas**.

25.3 Justificativas técnicas — 7,5%

Será observado se a equipe consegue justificar:

- PK — **1,5%**;
- FK — **1,5%**;
- restrições — **2,0%**;
- decisões arquiteturais relevantes — **2,5%**.

Pergunta central:

“POR QUE FOI FEITO ASSIM?”

25.4 Dados, BI e preparação para IA — 7,5%

Será observado se a equipe consegue:

- identificar informações estratégicas — **1,5%**;
- propor indicadores e métricas — **2,0%**;
- avaliar o potencial para BI — **2,0%**;
- justificar possibilidades analíticas e de IA — **2,0%**.

26. DIMENSÃO PROCEDIMENTAL — 50%

26.1 Modelo lógico — 10%

Critério	Peso
Tabelas	**2,0%**
Atributos	**1,5%**
PK	**2,0%**

FK **2,0%**
Restrições **2,5%**

26.2 Implementação SQL — 12%

Critério	Peso
Banco, tabelas e restrições	**4,0%**
Integridade referencial	**2,0%**
Massa de dados	**2,0%**
Consultas gerenciais	**2,0%**
Consultas de auditoria	**2,0%**

26.3 Documentação e repositório — 13%

Critério	Peso
Documentação e Dicionário de Dados	**4,0%**
Diagramas e scripts atualizados	**3,0%**
Histórico de commits	**3,0%**
Evidências colaborativas	**3,0%**

26.4 Apresentação e defesa final — 15%

A apresentação possuirá orientações próprias no **Manual de Apresentação e Defesa Final do Projeto**.

A avaliação da apresentação observará principalmente:

- compreensão do problema e da solução;
- capacidade de selecionar e defender decisões técnicas;
- demonstração de evidências da implementação;
- capacidade de propor evolução e agregar valor;
- comunicação, domínio e organização.

Não será necessário reapresentar todo o trabalho entregue.

A apresentação deverá demonstrar **compreensão e capacidade de defesa do projeto**.

27. DIMENSÃO ATITUDINAL — 20%

Participação e colaboração — 5%

- participação técnica — **2,5%**;
- colaboração — **2,5%**.

Responsabilidade — 5%

- cumprimento dos prazos — **2,5%**;
- entrega dos itens sob sua responsabilidade — **2,5%**.

Postura profissional — 5%

- comunicação profissional — **2,5%**;
- conduta ética e profissional — **2,5%**.

Autonomia e resolução de problemas — 5%

- busca independente de soluções — **2,0%**;
- proposição de melhorias — **3,0%**.

As evidências poderão ser obtidas ao longo do projeto por meio das atividades, diários de bordo, repositório, commits, entregas, acompanhamento do professor, avaliação 360° e defesa final.

28. O QUE NÃO FAZER

□ Começar diretamente pelo SQL

Primeiro valide o modelo.

□ Transformar cada entidade em tabela sem analisar os relacionamentos

Converter um modelo exige raciocínio.

□ Deixar relacionamentos N diretamente no modelo físico

Analise a necessidade de tabela associativa.

□ Dizer que normalizou sem demonstrar

Mostre as dependências e decisões relevantes.

□ Criar constraints sem saber por quê

Toda restrição deverá representar uma necessidade.

□ Criar consultas sem pergunta de negócio

Primeiro formule a pergunta. Depois escreva o SQL.

□ Criar uma massa de dados enorme e incoerente

Quantidade não substitui qualidade.

□ Utilizar IA e não validar o resultado

A responsabilidade pelo conteúdo entregue é da equipe.

□ Deixar DER, modelo lógico, dicionário e SQL diferentes entre si

Os artefatos precisam representar o mesmo projeto.

□ Concentrar todos os commits em um único integrante ou apenas no final

O histórico deverá demonstrar a evolução do projeto.

□ Deixar a documentação para depois de terminar o banco

Documentação faz parte do desenvolvimento.

29. TESTE DE CONSISTÊNCIA DA ENTREGA FINAL

Antes de entregar, percorram este caminho:

REQUISITOS

As necessidades da empresa continuam representadas?

↓

REGRAS DE NEGÓCIO

As regras estão claras e atualizadas?

↓

DER

Representa corretamente as regras?

↓

MODELO LÓGICO

Representa corretamente o DER?

↓

NORMALIZAÇÃO

As tabelas estão adequadamente estruturadas até a 3FN?

↓

DICIONÁRIO

Representa o modelo implementado?

↓

SQL

Implementa o modelo lógico?

↓

MASSA DE DADOS

Permite testar as regras e relacionamentos?

↓

CONSULTAS

Respondem a perguntas relevantes?

↓

BI/IA

Os dados possuem potencial analítico identificado?

↓

GITHUB

Outra pessoa consegue compreender e executar o projeto?

Se uma resposta for **NÃO**, revisem antes da entrega.

30. CHECKLIST FINAL DA EQUIPE

PRIMEIRA ENTREGA REVISADA

☐ Requisitos revisados

☐ Regras de negócio revisadas

☐ DER revisado

☐ Cardinalidades conferidas

☐ Alterações relevantes justificadas

MODELO LÓGICO

☐ Tabelas definidas

☐ Atributos definidos

☐ PKs definidas

☐ FKs definidas

☐ Chaves compostas verificadas

☐ Relacionamentos N tratados

☐ Entidades associativas verificadas

☐ Entidades fracas verificadas quando aplicável

☐ Restrições definidas

NORMALIZAÇÃO

☐ 1FN verificada

☐ 2FN verificada

☐ 3FN verificada

☐ Dependências funcionais relevantes documentadas

☐ Principais decisões de normalização justificadas

DICIONÁRIO DE DADOS

☐ Todas as tabelas documentadas

☐ Todos os atributos documentados

☐ Tipos de dados informados

☐ PKs identificadas

☐ FKs identificadas

☐ Restrições documentadas

☐ Dicionário coerente com o SQL

SQL

☐ CREATE DATABASE

☐ CREATE TABLE

☐ PRIMARY KEY

☐ FOREIGN KEY

☐ NOT NULL quando necessário

☐ UNIQUE quando necessário

☐ CHECK quando necessário

☐ DEFAULT quando necessário

☐ INSERT

☐ SELECT

☐ JOIN

☐ UPDATE

☐ DELETE

☐ Script executado e testado

MASSA DE DADOS

☐ Quantidade suficiente para testes

☐ Dados coerentes

☐ Relacionamentos testados

☐ Restrições testadas

☐ Prompts de IA utilizados na geração em escala documentados

CONSULTAS

☐ Consultas gerenciais implementadas

☐ Cada consulta responde a uma pergunta do negócio

☐ Consultas de auditoria implementadas

☐ Finalidade das consultas explicada

BI E IA

☐ Informações estratégicas identificadas

☐ Indicadores propostos

☐ Possibilidade de BI apresentada

☐ Possibilidade de IA apresentada

☐ Dados necessários identificados

☐ Benefício para a empresa explicado

GITHUB E DOCUMENTAÇÃO

☐ README atualizado

☐ DER atualizado disponível

☐ Modelo lógico disponível

☐ Dicionário de Dados disponível

☐ Normalização documentada

☐ Scripts organizados

☐ Consultas organizadas

☐ Histórico de commits preservado

☐ Evidências colaborativas disponíveis

☐ Todos os integrantes contribuíram

---

31. A PERGUNTA MAIS IMPORTANTE DA ENTREGA FINAL

Na primeira etapa, uma pergunta orientava o projeto:

“DE ONDE VEIO ESTA DECISÃO?”

Ela continua válida.

Mas agora acrescentaremos uma segunda:

“COMO VOCÊS PROVAM QUE ESTA DECISÃO FUNCIONA?”

Se alguém perguntar:

“Por que esta tabela existe?”

Vocês deverão saber responder.

“Por que esta FK está aqui?”

Vocês deverão saber responder.

“Por que utilizaram uma chave composta?”

Vocês deverão saber responder.

“Como sabem que esta tabela está na 3FN?”

Vocês deverão saber responder.

“Que regra esta constraint protege?”

Vocês deverão saber responder.

“Que informação esta consulta produz?”

Vocês deverão saber responder.

“Como esses dados poderiam apoiar uma decisão?”

Vocês deverão saber responder.

32. O QUE VOCÊS ESTÃO CONSTRUINDO?

Na primeira entrega, vocês transformaram:

UM PROBLEMA REAL

em:

UM MODELO CONCEITUAL DE DADOS.

Agora vocês deverão transformar:

MODELO CONCEITUAL

↓

MODELO LÓGICO

↓

MODELO NORMALIZADO

↓

MODELO FÍSICO

↓

BANCO IMPLEMENTADO

↓

DADOS

↓

INFORMAÇÃO

↓

APOIO À DECISÃO

A entrega final, portanto, não consiste apenas em “criar tabelas em SQL”.

O objetivo é demonstrar que a equipe consegue:

COMPREENDER → MODELAR → NORMALIZAR → IMPLEMENTAR → TESTAR → ANALISAR → JUSTIFICAR → EVOLUIR

Ao final, outra pessoa deverá ser capaz de acessar o repositório, compreender o problema, entender as decisões tomadas, executar o banco e perceber como os dados produzidos poderão gerar valor para a organização.

Prof. Clóvis